

《云计算与大数据处理》

实验报告

实验名称	云端数据接口与可复现环境（实验 01）
实验日期	2026 年 10 月 25 日
专业	
班级	
学号	2410311322
姓名	陶志杰
指导教师	

一、实验环境与身份

环境直接用课程给的 .venv，没有另外装 Python。这台机器上的实际版本是这样：

项目	实测内容
操作系统	Windows 11 家庭版 中文版
Python 版本	3.12.9
解释器路径	D:\mumu\work\.venv\Scripts\python.exe
项目工作目录	D:\mumu\work（课程根目录，所有命令都在这里执行）
boto3	1.43.93（AWS 官方 Python SDK，用来连本机的模拟服务）
moto	5.2.3（本机 S3 接口模拟器）
duckdb / pyarrow	1.5.5 / 25.0.1（这章没直接用到，后面几章共用）
依赖一致性检查	python -m pip check 输出 No broken requirements found
怎么调用解释器	不激活脚本，直接用 .venv\Scripts\python.exe 全路径，免得包装到系统 Python 上

依赖是按 requirements.lock.txt 装的，中途没有升级任何包。pip check 显示没有冲突。不过它只查包和包之间打不打架，跑不跑得通它管不了。



图 1　环境证据（Python 版本、解释器路径与关键包版本）

动手前先登记学号。这一步在课程里是硬性的，不登记的话，后面所有命令一开头就会停下。



图 2　学号登记证据（学号、实验/项目 01、run_id 与结果同屏）

二、问题与数据

2.1 要做的事

课程给了一个统一场景，叫模拟服务使用分析与规则查询平台。输入有两类，一类是合成的服务调用记录，一类是虚构的规则文档。这章只碰第一类。

要回答的问题很短：这些记录通过对象存储接口来回传一遍，还是原来那份吗？变了怎么发现？没变又说明得了什么？

先分清对象和文件。文件是磁盘上的一个路径，读写由本机文件系统负责；

对象是桶里的一份数据，由对象键、字节内容和元数据组成，通过 endpoint 指定的 HTTP 接口访问。对象键相当于对象的名字，不是磁盘路径，也不参与内容哈希。

2.2 数据怎么来的

数据是 coursekit seed --rows 20000 生成的。随机种子固定成 42，所以同样的代码、同样的参数跑两次，出来的文件一模一样。别人拿到结果，还能重新生成一遍。

字段	实测值	含义
valid_unique	20000	合法且唯一的事件
duplicates	100	把前 100 条又复制了一遍
invalid	2	故意造的坏行：bad-negative 的 input_tokens 是 -1，bad-time 的时间写成了 not-a-time
raw_rows	20102	原始 JSONL 总行数，20000 + 100 + 2
sha256	0d93bcd25e2804 7f804238c220bd b5b1d2dadf3286 47c07de85ba010 4e8e2854	原始文件的内容摘要
seed	42	固定种子，方便复现

原始文件在 data/raw/events.jsonl，20,102 行，一行一个 JSON 对象。每条

记录有 8 个字段：

字段	类型	这章的用途
event_id	字符串	事件的业务身份，后面用来去重
event_time	ISO8601 UTC 字符串	事件发生的时间，跟文件读取时间不是一回事
service	枚举 assistant/search/batch	按服务分组
model	枚举 small/medium	按模型分组
input_tokens / output_tokens	非负整数	用量，不是字数
latency_ms	非负整数	单次请求耗时
status	枚举 ok/error	成功还是失败

里面塞的 2 条坏记录我没删。删了之后，实验 02 就没法用 20102 = 20000 + 2 + 100 来说明清洗真的做过、而且没丢东西。

三、预测与核对

3.1 跑之前写的两条

两条预测是先写下来、存进 submissions/lab01/预测原稿.md 的，带时间戳。这样就不存在拿结果倒着填的问题。

预测的问题	我猜的	实际结果
同一份字节传到两个不同对象键，SHA-256 会变吗	会变	不变

预测的问题	我猜的	实际结果
传一个 input_tokens = -1 的文件会成功吗，能说明数据可用吗	会成功，但不能说明可用	会成功，也不能说明可用

3.2 第一条猜错了

实测跟我猜的相反。两个键不一样，摘要却都是23c0b20c65b4476c7e541044c09c88cd970bb02191681f804daef0b7ba49f0e8，一个字符都不差。

后来想明白了。SHA-256 只看文件里的字节。对象键只是对象在桶里的名字，算哈希的时候根本不参与。名字换了，内容没换，指纹自然不动。

反过来也一样。我把 payload 里的 tokens 从 120 改成 121，键没动，摘要立刻变成 1d0f0d28…。

第二条猜对了。上传返回 200，来回都成功。可 input_tokens 是负数，这跟传输没关系。所以上传成功只能说明网络通、接口调通、字节完整跑了个来回。它说明不了数据能不能用，也说明不了我真有云权限。

这么一拆，三件事就分开了。请求层看状态码。内容层看两个摘要一样不一样。业务层看字段合不合规。实验 01 只管中间那层。

四、运行结果

4.1 主流程

lab01 会在本机起一个临时 Moto 服务。接着建桶、开版本功能、把文件传两次、再下回本地，最后比摘要。回执如下：

项目	实测值
endpoint	http://127.0.0.1:65351（端口由系统随机分配）
桶 bucket	bdp-teaching
对象键 key	bronze/events/0d93bcd25e28047f804238c220bdb5b1d2dadf328647c07de85ba0104e8e2854.jsonl
字节数	3,892,184
上传前 source_sha256	0d93bcd25e28047f804238c220bdb5b1d2dadf328647c07de85ba0104e8e2854
下载后 download_sha256	0d93bcd25e28047f804238c220bdb5b1d2dadf328647c07de85ba0104e8e2854
hash_matches	true
version_ids_differ	true（同一个键连写两次，拿到两个不同的 VersionId）

两个判定都是 true。第一个说明字节一点没变。第二个说明同一个键写了两次，桶里存成了两个版本。版本号不一样，摘要一样。所以版本号记的是写过几次，摘要记的是内容是什么。



图 3　主流程回执（hash_matches 与 version_ids_differ 均为 true）

4.2 三次对照

附件脚本用的是内置的一小段 JSON，不碰 data/raw，所以可以放心反复跑

我跑了三次：

运行	对象键	内容 SHA-256	一致
基线	shared/raw/demo.json	23c0b20c65b4476c7e541044c09c88cd970b b02191681f804daef0b7ba49f0e8	true

运行	对象键	内容 SHA-256	一致
改对象键	students/2410311322/raw/demo.json	23c0b20c65b4476c7e541044c09c88cd970bb02191681f804daef0b7ba49f0e8（没变）	true
改 payload	students/2410311322/raw/demo.json	1d0f0d28612d93ec866d670eab8876e9dfb64d1c2b2cb84ee24fa60d0ee63bec（变了）	true

三个结果摆在一起，结论很清楚。改名字，摘要不动。改内容，摘要必变。

只要来回完整，下载一致性都成立。

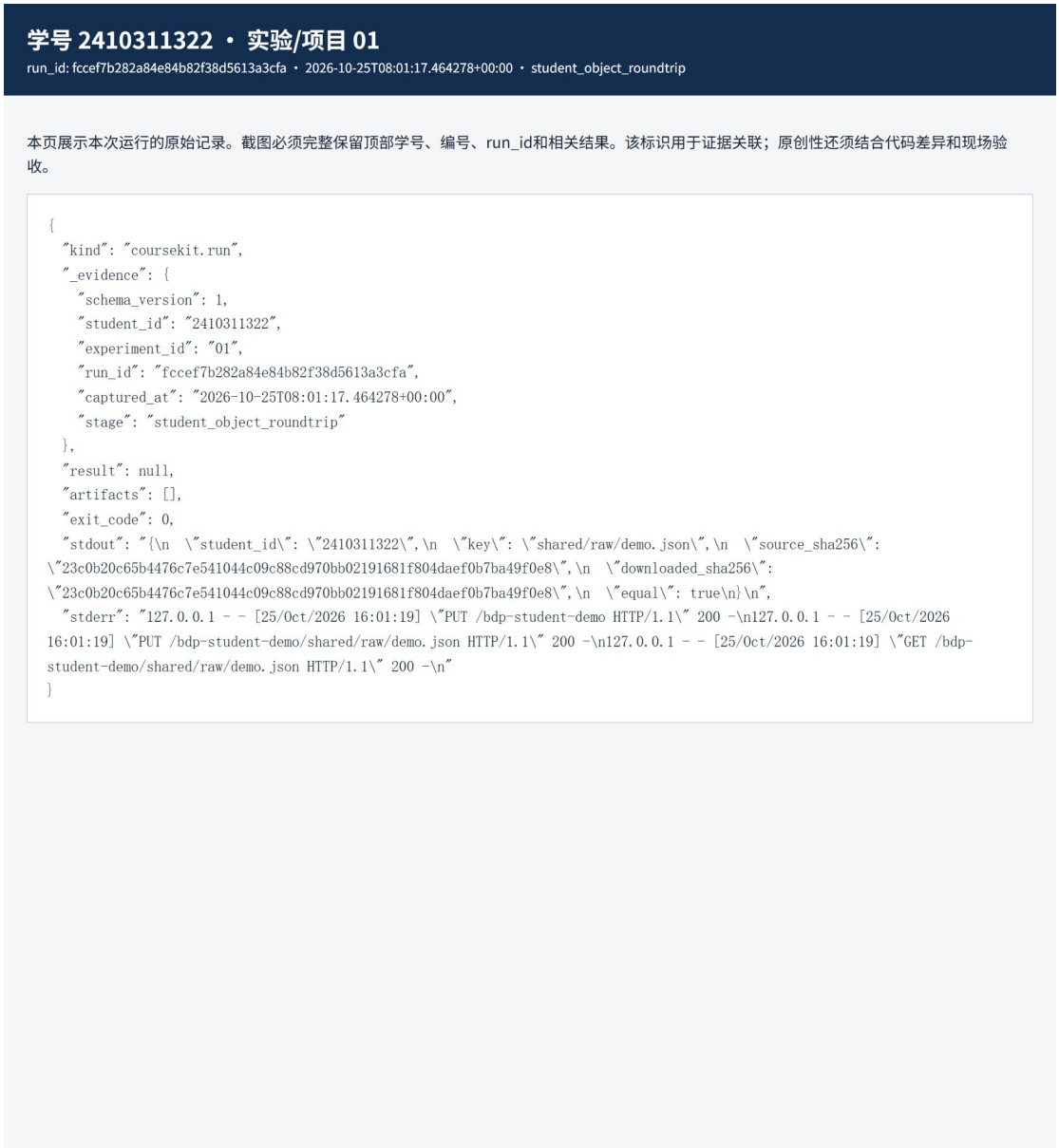


图 4　基线：对象键为 shared/raw/demo.json

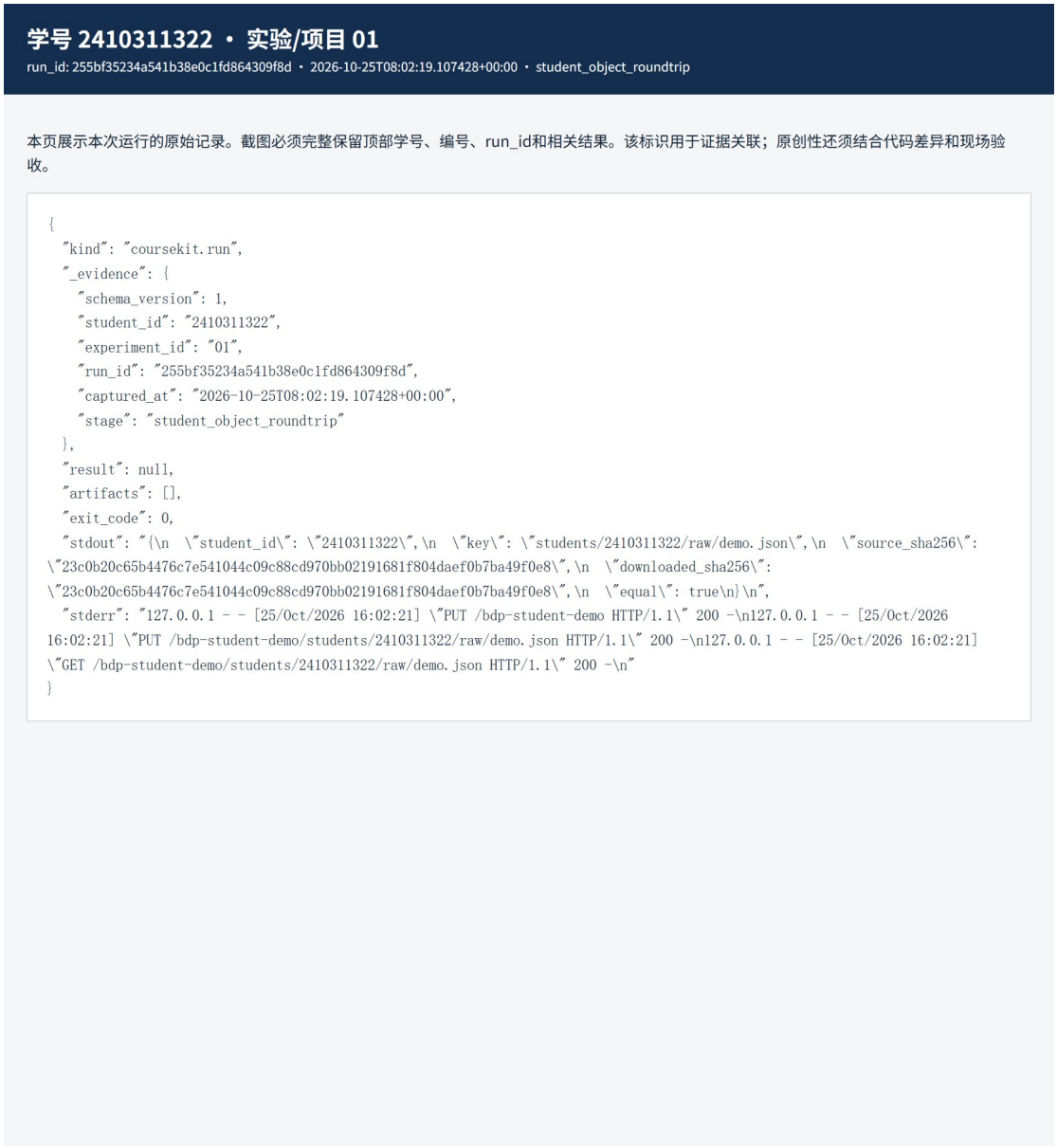


图 5　改对象键后：键已改变，内容摘要仍为 23c0b20c…



图 6　改 payload 后：键不变，摘要变为 1d0f0d28…

五、独立修改

5.1 改了什么

只动了一处。把 make_key() 里的公共前缀换成带自己学号的前缀，后缀不动。原附件留着没改，当作 diff 的参照。

```
def make_key():       # 独立改动：将共享前缀改为含自己学号的前缀，保留文件后缀。  -
```

```
return       "shared/raw/demo.json"       +                              return
"students/2410311322/raw/demo.json"
```

完整 diff 保存在 submissions/lab01/object_roundtrip.diff，原始附件留在课程目录，未改动。

payload 和哈希判定都没碰。

5.2 为什么要改

全班共用一个模拟桶。要是每个人的键都写 shared/raw/demo.json，谁后传谁覆盖前面的，桶里那份是谁的也说不清。加上学号，等于各占一格。

它只改名字，不改内容，所以摘要不动，就是上面第二行的结果。对象键负责找到东西，不负责校验。校验还得靠摘要。

5.3 第三项对照

我把 tokens 从 120 改成 121。跑之前猜两个摘要会变、键不变、下载一致还是 true。实测对上了，摘要变成 1d0f0d28612d93ec866d670eab8876e9dfb64d1c2b2cb84ee24fa60d0ee63bec。跑完我把 payload 改回 120，这样最后交的 diff 里只有 make_key 一处。

六、失败恢复

6.1 连不上服务

用 capture 包了一条故意失败的命令，地址指到本机 1 号端口：

```
.\venv\Scripts\python.exe -m coursekit capture --lab 01 --stage expected-failure -- .\.venv\Scripts\python.exe -m coursekit lab01 --endpoint http://127.0.0.1:1
```

报出来是这样：

```
EndpointConnectionError: Could not connect to the endpoint URL: "http://127.0.0.1:1/" exit_code: 1
```

1 号是低位端口，普通程序没权限在上面监听，所以一定连不上。这不是账号问题，也不是数据问题，就是连不上那个地址。失败和恢复都留了带学号和 run_id 的证据，没有把失败写成成功。

恢复的办法是去掉 --endpoint 参数，让它自己起临时服务。重跑之后 hash_matches 还是 true。



图 7　无效 endpoint 的真实失败记录（EndpointConnectionError，exit_code 1）

6.2 顺带撞上的第二个错

恢复的时候还碰到一个没预料到的问题：

```
FileExistsError: [WinError 183] 当文件已存在时，无法创建该文件。 'downloaded-events.jsonl.B8117Dae' -> 'downloaded-events.jsonl'
```

查源码才知道，这里用的是 boto3 的 download_file。它在 Windows 上先写临时文件，再改名到目标路径。如果 outputs/lab01/downloaded-events.jsonl 已

经存在，改名这一步就失败。所以 lab01 在这台机器上不能直接重跑。

我的处理是把旧的下载副本按 run_id 改名留着，再重跑，这才拿到新回执。这属于 Windows 上的操作限制，不是数据出错。就照实记下来，没去删整个 outputs 目录，也没改结果。

七、能力边界

整套流程跑在本机 Moto 上。它有真实的 HTTP 和 S3 语义，但不是公有云。下面逐项说清楚，没验证的就写没验证。

项目	这章的状态	依据
本地 HTTP 请求	验证	真实 SDK 往 127.0.0.1 发请求，服务端日志能看到 PUT/GET
S3 接口调用	验证	建桶、开版本、传两次、下载，都是真跑的
字节内容一致	验证	上传前后两个 SHA-256 完全相同
数据业务合法	未验证	这章只看字节；-1 的 token 和坏时间归实验 02
云耐久性	未验证	Moto 是内存模拟，服务一停对象就没了；回执里也写了 volatile memory
跨机容灾	未验证	只有本机一个进程，没有多机冗余或故障切换
真实访问控制	未验证	凭据是 teaching-only，模拟器不校验

这张表是用来圈定结论范围的。我能说接口真调用了、字节真没变。不能

说数据可用、云端持久、权限受控。后面这几项得有真实云环境、独立权限系统和多机故障注入才谈得上。

7.1 五道分析题

（1）同一份数据保存到两个不同对象键，SHA-256 不一定改变。哈希只计算对象内容，键是寻址信息，不进入摘要。实测两次上传的摘要完全相同。

（2）JSON 仅调整空格时，字节一致性和业务含义一致性不是同一件事。空格改变了字节，SHA-256 会变；解析后的字段和值可能完全相同，业务含义没变。字节校验和语义校验要分开说。

（3）上传一个 input_tokens=-1 的事件文件成功，只能证明传输链路和接口调用成功、字节完整到达，不能证明数据业务合法，也不能证明真实云权限或持久性。

（4）临时 Moto 服务停止后，实验 02 仍能开展，因为实验 02 读取的是本机 data/raw/events.jsonl 和清单，不依赖那个临时桶。Moto 只用于实验 01 的接口往返，数据在本地文件里。

（5）改用真实云对象存储，还要验证身份与权限（IAM、桶策略）、持久

性与版本行为、加密与合规、网络与区域、计费与配额、并发一致性。这些都不是本机 Moto 能给出的保证。

八、AI 使用披露

这次用了 AI 助手，用在三个地方：解释 boto3 和 Moto 的参数，帮忙找 6.2 那个重命名错误，整理报告的结构和排版。

预测是跑之前我自己写的。两处改动的判断也是我的。实测和预测对不上的地方，是我自己去核的。AI 说第一条预测有问题，我是看了两次运行的实际摘要才改的结论。

AI 给的说法我都拿终端输出和 storage.json 对过，对不上就不用。日志、哈希、截图，没有一处是 AI 生成的。

九、总结与交接

这一章做完的事：环境弄成可复现的；用固定种子生成 20,102 行数据；通过真实 S3 接口上传、下载、比摘要；改了一处对象键，用两次对照说明键变摘要不变；用无效地址制造一次连接失败，再恢复。

结论有三条。摘要只能说明字节一致，说明不了数据可用，也说明不了来源可信。对象键是定位用的，不承担校验。同一个键写多次会得到不同版本号，版本号和摘要别混着用。

9.1 交给实验 02 的东西

交接内容	路径	下游怎么用
原始事件文件	data/raw/events.jsonl	实验 02 的清洗输入，20,102 行
原始清单	data/manifests/raw.json	记录规模、摘要和生成参数
往返回执	outputs/lab01/storage.json	实验 02 启动时拿这里的 source_sha256 跟当前文件比对
下载副本	outputs/lab01/downloaded-events.jsonl	留着当字节一致的旁证

9.2 还没处理的问题

有几个问题这次故意留着。100 条重复会让用量统计偏高。bad-negative 里的 -1 如果当成退款，统计会偏低。bad-time 混进按日期分组，日统计会失真。同一个 ID 但内容不一样时该怎么更正，也还没定。

另外要注意，实验 02 一启动就会拿 outputs/lab01/storage.json 里的 source_sha256 跟当前 data/raw/events.jsonl 比对。对不上就直接拒绝往下走。所

以这章的原始文件得留着，不能为下一章重新生成一份随机数据。

9.3 运行记录索引

报告里每张图对应的运行记录如下。按 run_id 可以在 evidence/ 里找到原始 JSON 和同名 HTML。

图	run_id
图 1	20e8ed6a426e4a99b189c6fdfc9af52d
图 2	ed66cb5f96ef447d8dffff1d7620ea6d
图 3	078a37e0cb2d4afda9106f553b42817b
图 4	fccef7b282a84e84b82f38d5613a3cfa
图 5	255bf35234a541b38e0c1fd864309f8d
图 6	2b14c3e59cf34acb822859bf250a7240
图 7	035558819c5f4641b9440554a6aa7844